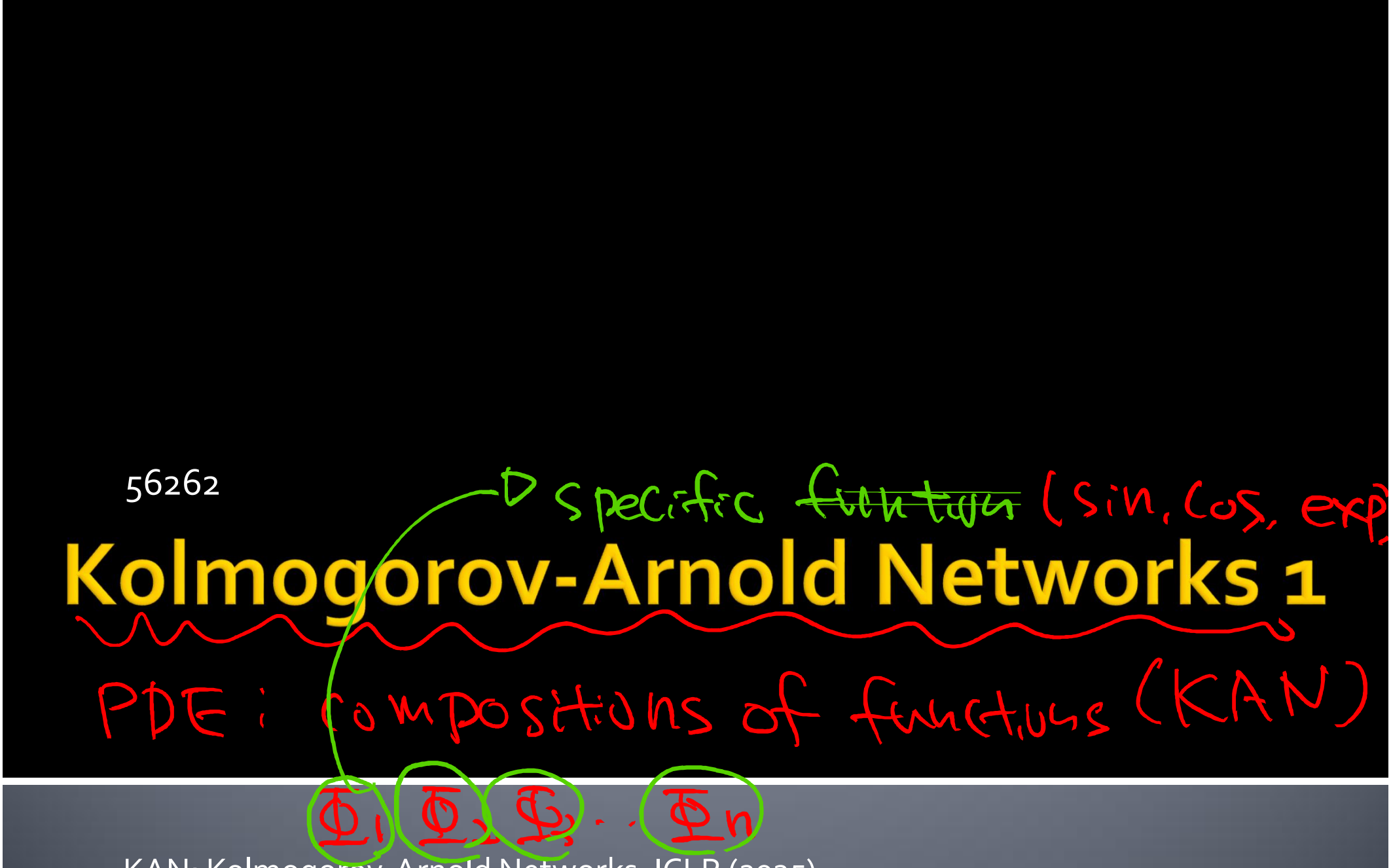
56262

Kolmogorov-Arnold Networks 1



KAN: Kolmogorov-Arnold Networks, ICLR (2025)

Basic Idea of KAN

- **Basic Idea**
  - High-dimensional functions $y = f(x)$ can be decomposed into compositions of 1D functions

$$f(x) = \sum_q \Phi_q \left(\sum_p \phi_{q,p}(x_p) \right)$$

n-dim = sum of 1-dim

  - These 1D functions are learnable (e.g., spline)

- **Advantages**
  - Interpretability : Each function can be inspected
  - Accuracy : Better approximation of structured (PDE) functions
  - Better scaling laws : Faster improvement with model size

MLP vs. KAN 1/2

- **Core Architectural Shift**
  - Each "weight" is replaced by a learnable 1D function (e.g., spline)

	MLP	KAN
Diff 1	Activation on nodes (Fixed)	Activation on edges (Learned)
Diff 2	Weights are scalars	Weights are functions
Diff 3	Nodes apply nonlinearity	Nodes just sum inputs

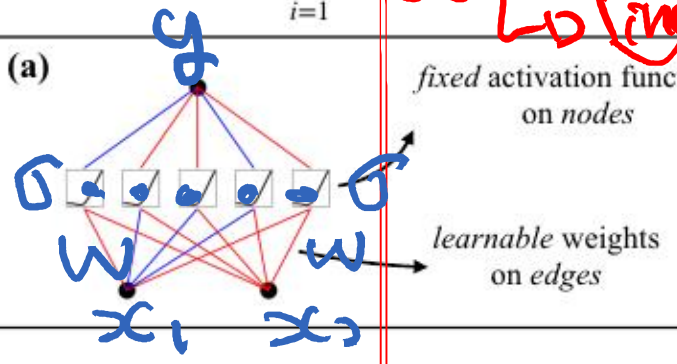
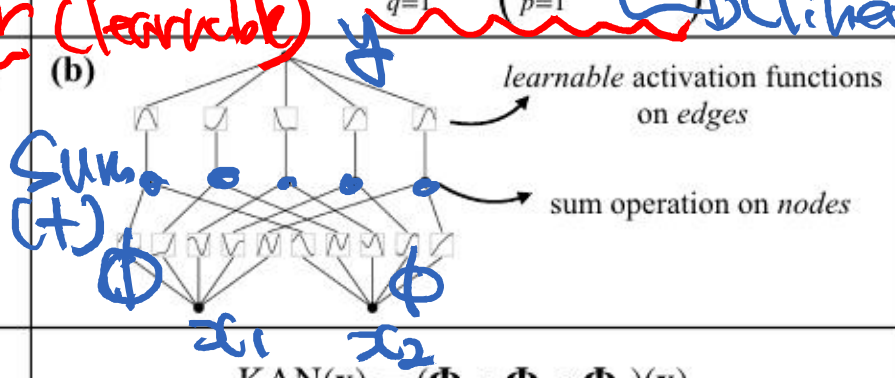
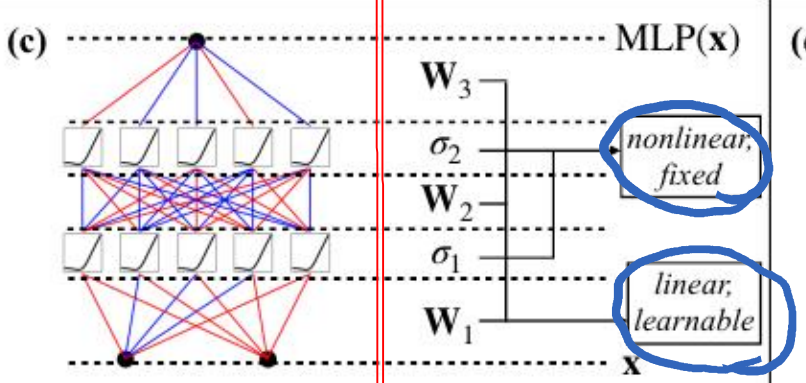
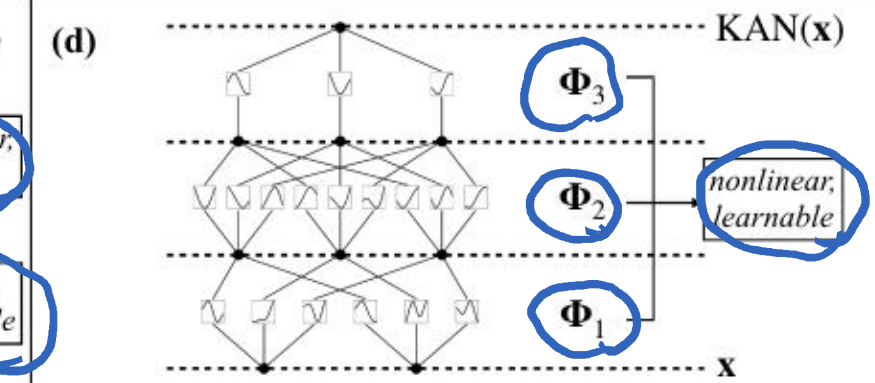
  - KAN combines two complementary strengths: "**MLPs** on the outside, **Splines** on the inside"

	Splines	MLPs
Pros	Highly accurate in low dimensions (local)	Handle high-dimensional composition well (global)
Cons	But suffer from curse of dimensionality	But are less precise locally

MLP vs. KAN 2/2

MLP: $x \to Wx \to \sigma(\cdot)$

KAN: $x \to \phi(x) \to \text{sum}$

Model	Multi-Layer Perceptron (MLP)	Kolmogorov-Arnold Network (KAN)
Theorem	Universal Approximation Theorem	Kolmogorov-Arnold Representation Theorem
Formula (Shallow)	$f(\mathbf{x}) \approx \sum_{i=1}^{N(\epsilon)} a_i \sigma(\mathbf{w}_i \cdot \mathbf{x} + b_i)$	$f(\mathbf{x}) = \sum_{q=1}^{2n+1} \Phi_q \left(\sum_{p=1}^{n} \phi_{q,p}(x_p) \right)$
Model (Shallow)		
Formula (Deep)	$\text{MLP}(\mathbf{x}) = (\mathbf{W}_3 \circ \sigma_2 \circ \mathbf{W}_2 \circ \sigma_1 \circ \mathbf{W}_1)(\mathbf{x})$	$\text{KAN}(\mathbf{x}) = (\mathbf{\Phi}_3 \circ \mathbf{\Phi}_2 \circ \mathbf{\Phi}_1)(\mathbf{x})$
Model (Deep)		

Kolmogorov–Arnold Representation Theorem

- **Theorem**
  - High-dimensional functions can be decomposed into 1D functions + addition
  - Any multivariate function can be written as:
    - sums of
    - compositions of **univariate (1D) functions**
- **Limitation of Theorem**
  - These 1D functions can be non-smooth or fractal
  - Hard to learn in practice

KAN Architecture 1/3

- **Layer**
  - A layer is defined as a matrix of functions:

$$\Phi = \{\phi_{q,p}\} \qquad \begin{matrix} p = 1, 2, \cdots, n_{\text{in}}, \\ q = 1, 2 \cdots, n_{\text{out}}. \end{matrix}$$

    - input dimension: n_{in}
    - output dimension: n_{out}

- **Edge**
  - Each connection has a function (parameterized as **B-splines**):

$$\phi_{q,p} : \mathbb{R} \to \mathbb{R}$$

    - takes a scalar input
    - outputs a transformed scalar

KAN Architecture 2/3

- **Forward Pass**
  - Each neuron computes:

$$x_{l+1,j} = \sum_{i=1}^{n_l} \phi_{l,j,i}(x_{l,i})$$

Nodes = **aggregation only**

Edges = **nonlinearity**

1. Each input value passes through a function (edge)
2. All outputs are summed at the node

(l+1)-th layer data

l-th layer data

  - Matrix Form $x_{l+1} = \Phi_l(x_l)$

matrix of functions

- **Full Network**

$$KAN(x) = (\Phi_{L-1} \circ \cdots \circ \Phi_0)(x)$$

KAN Architecture 3/3

- **Limitation of Theorem**
  - These 1D functions can be non-smooth or fractal
  - Hard to learn in practice
- **KAN uses B-splines because:**
  - They approximate 1D functions well
  - They are smooth
  - They have local support

KAN replaces scalar weights with learnable 1D functions, forming layers as function matrices. Nodes only sum inputs, and the model is inspired by the Kolmogorov–Arnold theorem that decomposes multivariate functions into compositions of univariate ones.

Approximation Theory

- **Core Statement**
  - KAN can approximate multivariate functions with a convergence rate that does not depend on the input dimension.
- **Intuition**
  - Standard neural networks (MLPs): $\text{Error} \sim O(N^{-k/d})$
  - KAN $\text{Error} \sim O(N^{-k})$
    - N: number of parameters
    - d: input dimension

data dimension X

data dimension

- KAN reduces a high-dimensional approximation problem into many low-dimensional ones (1D using splines), which have strong approximation properties in 1D:.
- KAN avoids the curse of dimensionality by achieving dimension-independent approximation rates.

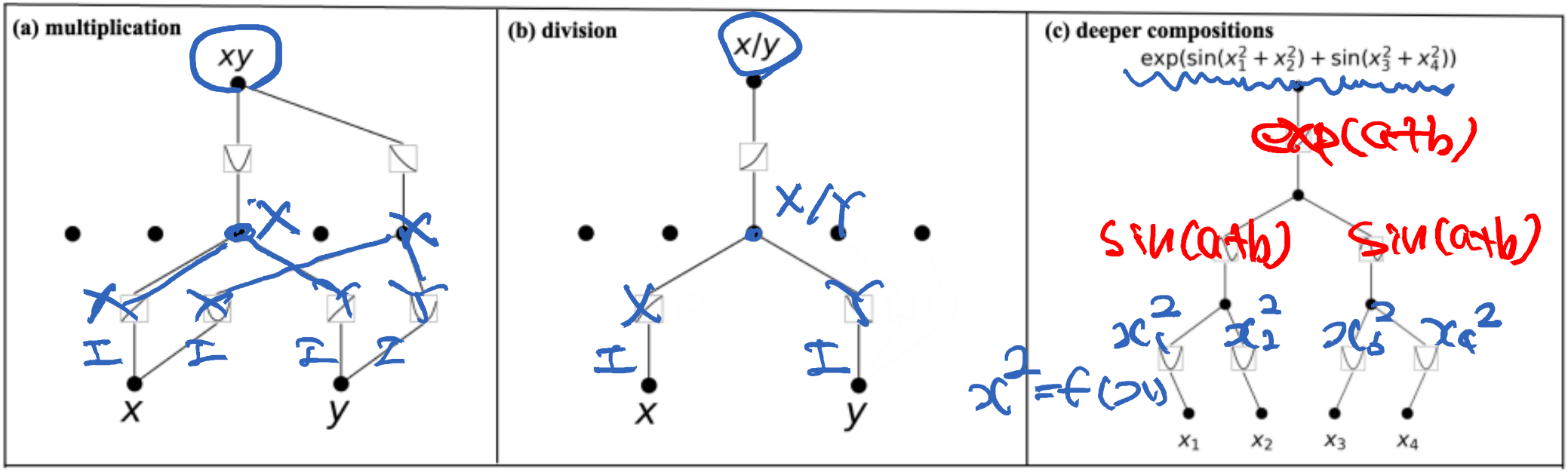
Interpretability in KAN 1/2

- **Symbolification**
  - Convert learned functions into human-readable mathematical expressions.
  - KAN learns functions as splines. After training:

1. Each spline function is approximated using a set of candidate functions:
   - linear
   - polynomial
   - sinusoidal
   - exponential, etc.
2. The best-fitting symbolic form is selected.

KAN can turn the model into an explicit mathematical formula.

Interpretability in KAN 2/2



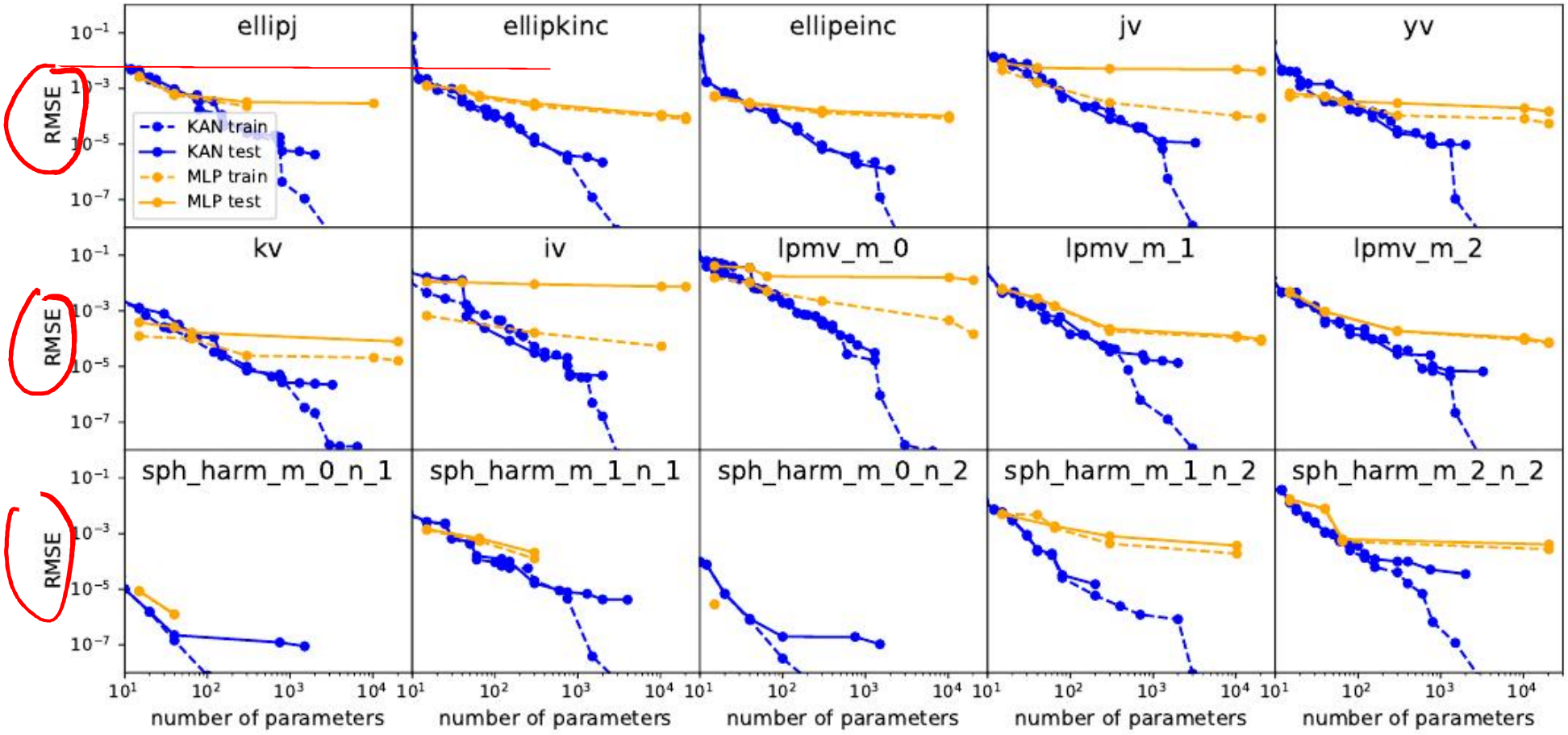
Function Fitting 1/2

- **Fit synthetic functions :** 15 special functions common in math and physic

Name	scipy.special API	Minimal KAN shape test RMSE $< 10^{-2}$	Minimal KAN test RMSE	Best KAN shape	Best KAN test RMSE	MLP test RMSE
Jacobian elliptic functions	$\mathrm{ellipj}(x, y)$	[2,2,1]	7.29×10^{-3}	[2,3,2,1,1,1]	$\mathbf{1.33 \times 10^{-4}}$	6.48×10^{-4}
Incomplete elliptic integral of the first kind	$\mathrm{ellipkinc}(x, y)$	[2,2,1,1]	1.00×10^{-3}	[2,2,1,1,1]	$\mathbf{1.24 \times 10^{-4}}$	5.52×10^{-4}
Incomplete elliptic integral of the second kind	$\mathrm{ellipeinc}(x, y)$	[2,2,1,1]	8.36×10^{-5}	[2,2,1,1]	$\mathbf{8.26 \times 10^{-5}}$	3.04×10^{-4}
Bessel function of the first kind	$\mathrm{jv}(x, y)$	[2,2,1]	4.93×10^{-3}	[2,3,1,1,1]	$\mathbf{1.64 \times 10^{-3}}$	5.52×10^{-3}
Bessel function of the second kind	$\mathrm{yv}(x, y)$	[2,3,1]	1.89×10^{-3}	[2,2,2,1]	$\mathbf{1.49 \times 10^{-5}}$	3.45×10^{-4}
Modified Bessel function of the second kind	$\mathrm{kv}(x, y)$	[2,1,1]	4.89×10^{-3}	[2,2,1]	$\mathbf{2.52 \times 10^{-5}}$	1.67×10^{-4}
Modified Bessel function of the first kind	$\mathrm{iv}(x, y)$	[2,4,3,2,1,1]	9.28×10^{-3}	[2,4,3,2,1,1]	$\mathbf{9.28 \times 10^{-3}}$	1.07×10^{-2}
Associated Legendre function $(m = 0)$	$\mathrm{lpmv}(0, x, y)$	[2,2,1]	5.25×10^{-5}	[2,2,1]	$\mathbf{5.25 \times 10^{-5}}$	1.74×10^{-2}
Associated Legendre function $(m = 1)$	$\mathrm{lpmv}(1, x, y)$	[2,4,1]	6.90×10^{-4}	[2,4,1]	$\mathbf{6.90 \times 10^{-4}}$	1.50×10^{-3}
Associated Legendre function $(m = 2)$	$\mathrm{lpmv}(2, x, y)$	[2,2,1]	4.88×10^{-3}	[2,3,2,1]	$\mathbf{2.26 \times 10^{-4}}$	9.43×10^{-4}
spherical harmonics $(m = 0, n = 1)$	$\mathrm{sph_harm}(0, 1, x, y)$	[2,1,1]	2.21×10^{-7}	[2,1,1]	$\mathbf{2.21 \times 10^{-7}}$	1.25×10^{-6}
spherical harmonics $(m = 1, n = 1)$	$\mathrm{sph_harm}(1, 1, x, y)$	[2,2,1]	7.86×10^{-4}	[2,3,2,1]	$\mathbf{1.22 \times 10^{-4}}$	6.70×10^{-4}
spherical harmonics $(m = 0, n = 2)$	$\mathrm{sph_harm}(0, 2, x, y)$	[2,1,1]	1.95×10^{-7}	[2,1,1]	$\mathbf{1.95 \times 10^{-7}}$	2.85×10^{-6}
spherical harmonics $(m = 1, n = 2)$	$\mathrm{sph_harm}(1, 2, x, y)$	[2,2,1]	4.70×10^{-4}	[2,2,1,1]	$\mathbf{1.50 \times 10^{-5}}$	1.84×10^{-3}
spherical harmonics $(m = 2, n = 2)$	$\mathrm{sph_harm}(2, 2, x, y)$	[2,2,1]	1.12×10^{-3}	[2,2,3,2,1]	$\mathbf{9.45 \times 10^{-5}}$	6.21×10^{-4}

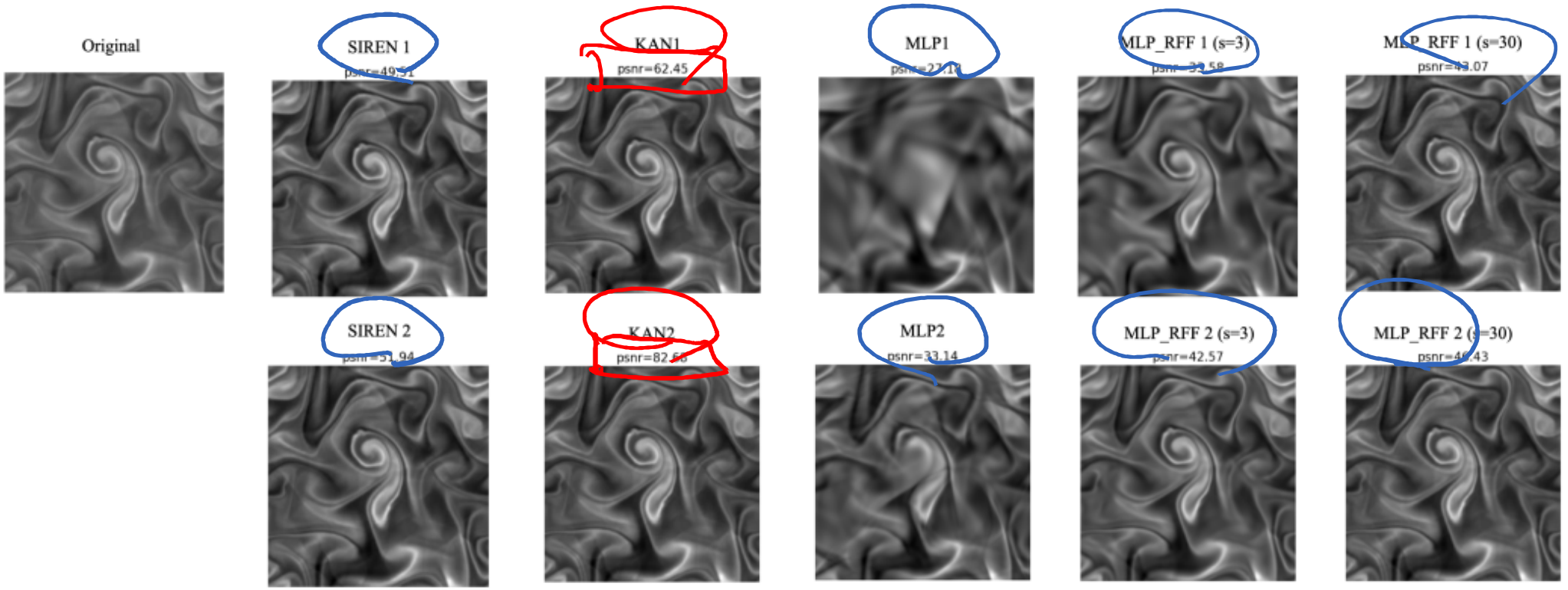
Function Fitting 2/2

- KAN matches the underlying structure of the function



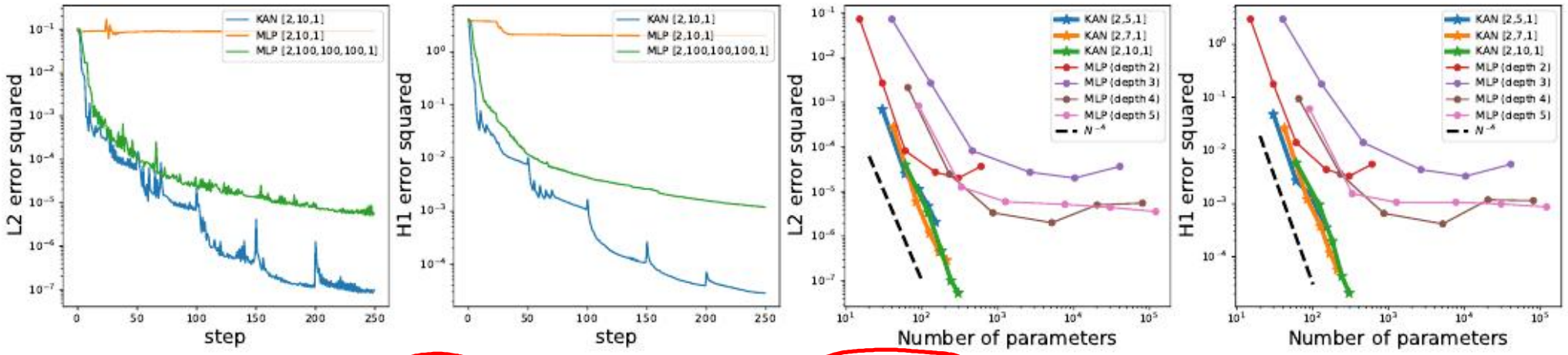
KAN uses parameters more efficiently due to 1D decomposition

Fitting Images



Partial Differential Equations

- Scientific functions are naturally compositional → KAN matches inductive bias



Poisson equation with zero Dirichlet boundary data

KAN Loss +PDE Loss